

一、项目定位与 MVP 定义

1. 项目一句话定义

面向高校的个性化学习资源生成与辅导系统，利用多智能体架构和检索增强生成（Agentic RAG）技术，以科大讯飞星火大模型为核心，实现学生画像、知识检索、防幻觉回答、多模态资源生成和学习路径规划的闭环服务。

2. 最终成品形态

一个可以部署在私有服务器或云端的 Web 应用（支持桌面和移动端），系统提供对话式学习助手界面，旁边展示思维导图、PPT 下载链接、测验卡片等多模态内容。教师可上传课程资料并自动构建知识库；学生可通过对话获取针对性学习资源，并进行在线测验和学习路径推荐。

3. 核心用户

主要用户包括：

1. **高校学生** —— 需要根据自己的基础和兴趣获取个性化、多模态学习资源；
2. **教师** —— 上传课程资料并查看学生画像与学习反馈；
3. **教务管理者**（可选） —— 查看全局数据、配置系统参数。

4. 核心使用场景

- **学生提问与辅导**：学生通过聊天界面提出问题，系统采用多智能体协作流程拆解问题、检索知识库并生成带出处的回答，同时在右侧展示思维导图或流程图。
- **个性化资源生成**：根据学生画像和课程知识点自动生成 PPT、思维导图、测验题、练习案例等资源；学生可下载或在线查看。
- **学习路径推荐**：结合学生掌握度和认知风格，自动规划下一步学习内容并推送相应资料。
- **教师资料上传**：教师上传课程教材 PDF/Word 文档，系统自动切分并构建向量索引和知识图谱。

5. 必须实现的功能

功能	具体要求
学生画像构建	通过对话收集学生的专业、学习目标、历史成绩、认知风格、学习节奏等信息，并存储为结构化画像数据；系统要利用自然语言对话隐式提取这些特征，至少支持六个维度的学习画像。
Agentic RAG 问答	采用 LangGraph + 星火大模型编排的多智能体流程，对学生问题进行深度意图分析、子问题拆解、检索、验证和生成，输出带引用的答案，防止大模型幻觉。
课程知识库构建	教师上传的资料通过 PDF 版面分析和命名实体识别等方法拆分为知识块并构建向量索引；支持在知识库中存储先决关系图谱。

功能	具体要求
个性化资源生成	支持自动生成五种类型资源：课程讲解文档（文本形式）、思维导图（Mermaid 代码渲染）、练习题（JSON 格式）、拓展阅读推荐、简单 PPT；借助 Presenton、Mermaid.js 等工具。
学习路径规划	根据知识图谱和学生画像，计算最短学习路径并推荐下一步内容。
前端多模态展示	利用 LobeChat 框架提供的 Artefacts 和多模态卡片功能，将文本、PPT 链接、思维导图和测验卡片分别渲染于不同区域。
用户管理与登录	支持基本用户注册、登录、身份校验及角色管理（学生、教师、管理员）。
知识检索 API	提供给前端或第三方应用的查询接口，返回引用文本和源文件路径。
部署与安全	私有化部署，遵循科大讯飞 API 使用规范；具备内容安全过滤功能。

6. 可选加分功能

- **智能辅导与答疑视频**：通过科大讯飞 Sparkgen 或 Seedance 2.0 生成数字人视频或复杂实验动画；
- **学习效果评估**：动态跟踪学生练习情况，生成阶段报告，并反馈到学生画像；
- **情感计算与语音交互**：利用 LobeChat 内置的 TTS/STT 支持语音输入输出；
- **数据可视化后台**：为教师和管理员提供对知识库覆盖度、学生画像统计和使用情况的仪表盘。

7. 不建议优先做的功能

- **高保真视频生成**：Seedance 2.0 虽支持 2K 60FPS 多镜头视频生成，但训练和推理成本高、不易在短期部署，应作为后期扩展；
- **复杂 3D 仿真与物理引擎**：项目初期不必实现复杂的物理仿真动画，可用静态图或简短演示替代；
- **完全自动化知识图谱构建**：初赛阶段可依赖简单的 PDF 切分和向量检索；真正构建精细知识图谱需要投入较多时间，可逐步引入 K12EduKG 等工具。

8. 最小可行版本 MVP

MVP 阶段只实现对话问答、基本知识库检索和思维导图生成三大闭环，并以 LobeChat 作为前端和聊天入口，LangGraph + 星火大模型作为智能体编排核心；使用轻量向量库 ChromaDB 存储文本切片；采用 Mermaid.js 生成思维导图；暂不做视频、音频和完整 PPT 生成。

二、开源项目筛选表

表格中列出了报告中涉及的主要开源项目、框架和工具，并给出推荐级别及使用建议。

名称	GitHub/官网	所属模块	主要作用	推荐程度	推荐理由与风险	集成难度	放置位置	是否二次
LangGraph	https://github.com/langchain-ai/langgraph	Agent 编排	基于有向无环图的智能体流程编排，支持显式状态管理	强烈推荐	图结构可控制性强、与 LangChain 生态兼容、适合教学流程拆解；风险在于对图模型理解要求高	中	Agent 编排层	需编点函状态
CrewAI	https://github.com/joaoimdmoura/crewai	Agent 框架	角色任务驱动的多智能体框架	可用	学习曲线平缓，适合团队协作模拟；但难以精细控制指令执行顺序	中	备选 Agent 层	可能整 Prom 模板
AutoGen	https://github.com/microsoft/autogen	Agent 框架	对话驱动的智能体框架	只参考	适合探索性研究，但对话不可控且易陷入死循环	中	不建议用于主系统	需要二次
Astron Agent	https://astronagent.com	Agent 平台	企业级工作流平台，集成视觉编排和 RPA	不建议	依赖度高，功能重，不适合学生团队快速落地	高	外部 SaaS	无
Spark LLM (科大讯飞星火大模型)	https://xinghuo.xfyun.cn	核心推理引擎	文本和多模态生成模型，支持多智能体协作	强烈推荐	与科大讯飞生态兼容，提供 Agentic RAG 和多模态生成能力；官方要求使用，API 稳定	低	服务端外部 API	无需开发需封 API
K12EduKG / PDFMiner	https://github.com/PaddlePaddle/K12EduKG / https://github.com/pdfminer/pdfminer.six	知识图谱构建	用于解析教材 PDF 并提取知识点，构建知识图谱	可用	K12EduKG 提供教育知识图谱样板；PDFMiner 用于版面分析；但构建完整图谱成本高	高	知识库层	需要化开语义挖掘
Presenton	https://github.com/presenton-ai/presenton	多模态生成	私有化 AI PPT 引擎，输入 Markdown 输出 PPTX、PDF	强烈推荐	Apache 2.0 协议，支持离线部署，适合生成课程 PPT；风险是模板风格有限	中	多模态生成层	通过 RES 调用封装

名称	GitHub/官网	所属模块	主要作用	推荐程度	推荐理由与风险	集成难度	放置位置	是否二次
Mermaid.js	https://github.com/mermaid-js/mermaid	多模态生成	浏览器端渲染思维导图、流程图，支持 Markdown 语法	强烈推荐	轻量级、渲染效果好，学生可编辑，前端即可渲染	低	前端与多模态层	不需
Seedance 2.0	https://github.com/Seedance/Seedance	视频生成	高质量多模态视频生成模型	只参考	输出效果惊艳，但计算资源要求高；可用作演示视频	高	多模态生成层	调用杂，GPU 持
Exam-Quiz-Test-PRO	https://github.com/victordarras/exam-quiz-test-pro	测验生成	HTML/JS 测验生成器，用于前端容器	可用	可快速生成答题界面并支持 JSON 题库输入	低	多模态生成层	无需
LobeChat (LobeHub)	https://github.com/lobehub/lobe-chat	前端框架	开源多模态聊天 UI，支持 Artifact 渲染、卡片化内容	强烈推荐	自带 Spark API 集成、TTS/STT、Artifact 视图等；极大降低前端开发成本	低	前端层	无需求，凭证
ChromaDB	https://github.com/chroma-core/chroma	向量数据库	轻量向量数据库，支持持久化嵌入存储	强烈推荐	容易部署、Python 接口友好，适合存储教材切片；MVP 阶段足够	低	RAG 层	仅需 API；索引
Neo4j	https://neo4j.com	图数据库	存储学习画像和知识图谱，支持图查询	可用	适合存储先决关系和画像动态更新；需要部署和学习 Cypher 语法	中	用户画像层	需要与查数开
FastAPI	https://fastapi.tiangolo.com	后端框架	高性能异步 Web 框架，用于 REST API 和后台服务	强烈推荐	Typing 友好、性能高，与 Python 生态良好；适合快速编写接口和异步任务	低	后端 API 层	自写和依入
Celery/Redis Queue	https://docs.celeryq.dev/	异步任务队列	支持后台生成任务异步执行	可用	用于 PPT 生成、视频生成等长任务；配置稍复杂	中	多模态层 / Worker	需要任务与调

名称	GitHub/官网	所属模块	主要作用	推荐程度	推荐理由与风险	集成难度	放置位置	是否二次
iFlyCode IDE 插件	https://developer.xfyun.cn	辅助工具	科大讯飞智能编程助手，提供自动补全和测试生成	可用	加速开发，但不是系统必需	低	开发工具	无需成，开发装
Dify / FastGPT	https://github.com/langgenius/dify/ https://github.com/flashcat-ai/fastgpt	AI 应用平台	集成 LLM、RAG、Agent 可视化 管理	只参考	功能强大，但部署复杂且不完全兼容星火模型；适用于企业级扩展	高	可选平台	无

最终推荐技术组合

综合上述表格和项目要求，建议的核心技术组合如下：LangGraph（Agent 编排）+ Spark LLM（推理引擎）+ ChromaDB（向量库）+ FastAPI（后端）+ LobeChat（前端）+ Presenton + Mermaid.js。其他项目视需要辅助引入。

三、最终推荐技术栈

下表列出了各技术环节的推荐方案，并结合“是否必须”与“是否有替代方案”给出分析。

环节	推荐技术	选择理由与备注	替代方案	Demo 阶段必需?	正式版本保留?
前端框架	LobeChat (Next.js + React)	前端开发量极小，原生支持与星火模型对接、多模态卡片、Mermaid 渲染，用户体验优异	Chatbot UI、Vue3 + Element UI	是	是
UI 组件库	LobeChat 内置 Ant Design 主题	LobeChat 已集成 Ant Design 风格的卡片和交互，无需额外 UI 库	Ant Design Vue/React	是	是
后端框架	FastAPI (Python)	异步支持好，类型提示完备，易与 LangChain/LangGraph 集成；Starlette 架构性能高	Django Rest Framework	是	是
Agent 框架	LangGraph	图结构控制流适合教学流程并保证可解释性	CrewAI	是	是
RAG 框架	LangChain + ChromaDB	LangChain 提供文档加载、切分和检索封装；ChromaDB 轻量易部署	LlamaIndex、Vespa	是	是

环节	推荐技术	选择理由与备注	替代方案	Demo 阶段必需?	正式版本保留?
向量数据库	ChromaDB	本地运行无需额外服务，部署简单，适合初赛	Weaviate、Milvus	是	视规模而定
关系型数据库	PostgreSQL 或 SQLite	用于存储用户、课程、生成记录；PostgreSQL 功能强，SQLite 适合本地 demo	MySQL	是	是
图数据库	Neo4j	用于存储学生画像和知识图谱关联	ArangoDB	可选	视后期需求
文件存储	Minio 或本地目录	存放上传的教材、生成的 PPT/视频等；Minio 兼容 S3 协议	AWS S3、Azure Blob	是	是
大模型 API	科大讯飞 Spark LLM	赛题要求使用，支持多智能体协作与多模态生成	无	是	是
语音模块	LobeChat 内置 TTS/STT	适合语音输入输出和无障碍功能	科大讯飞开放平台接口	可选	可选
图表与 PPT 生成	Mermaid.js、Presenton	Mermaid.js 生成思维导图；Presenton 生成 PPT	pptxgenjs	是	是
登录认证	JWT（通过 FastAPI 实现）	轻量、安全，易于前后端分离	OAuth2	是	是
部署方案	Docker Compose	快速搭建多服务环境；结合 Nginx 代理和 HTTPS；本地或云端均可部署	Kubernetes	是	视后期规模
日志与监控	Python logging + Grafana/Prometheus	在 Demo 阶段用 logging 即可，正式版可接入监控平台	ELK Stack	可选	可选
开发工具	iFlyCode IDE 插件	提高开发效率，自动生成样板代码	GitHub Copilot	可选	可选

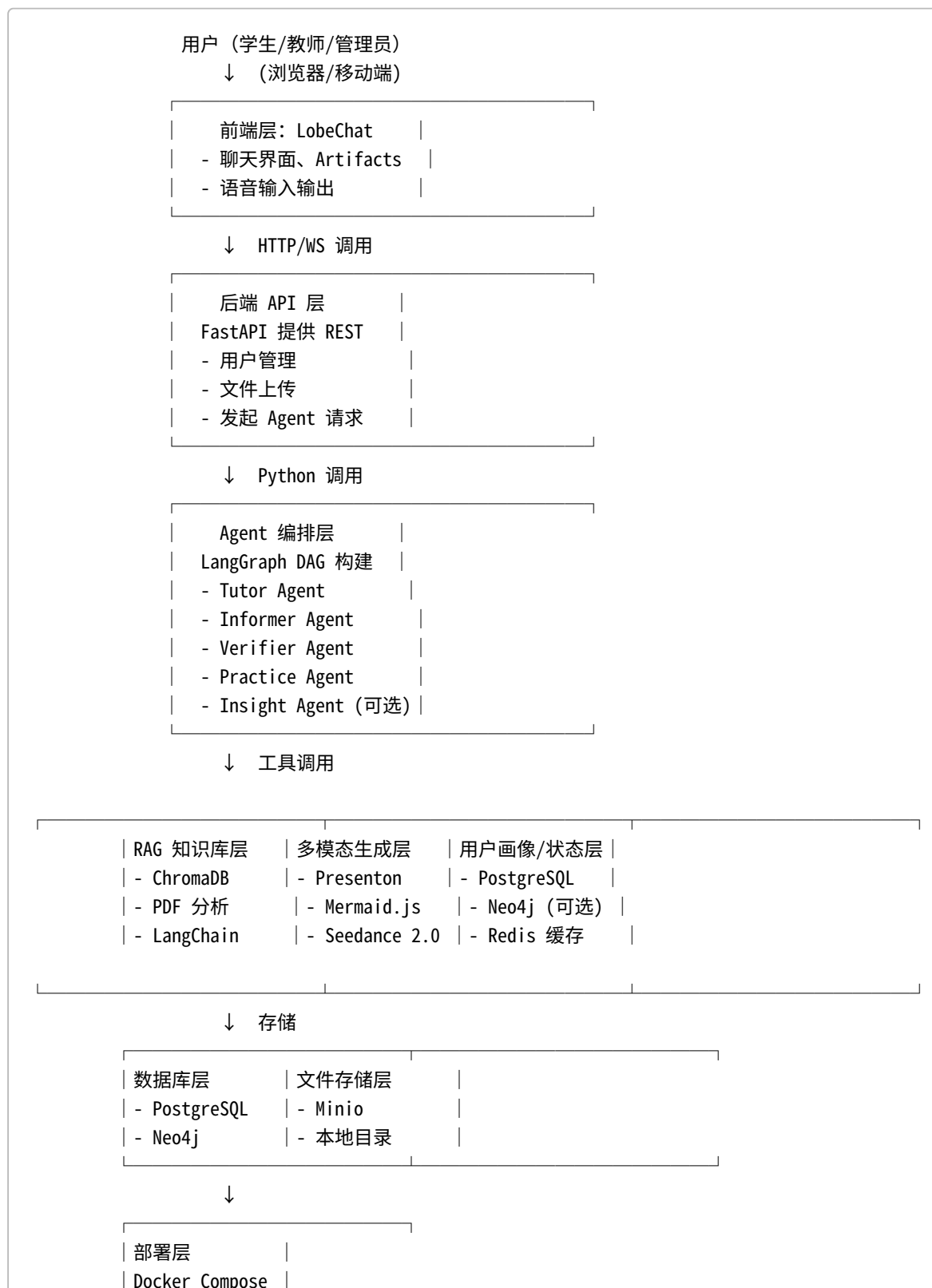
技术栈分析

- **前端与 UI**：LobeChat 将复杂的多模态展示功能抽象为可插拔组件，适合团队直接部署、快速迭代，避免重复造轮子；其内置的 Spark API 集成省去开发鉴权与请求解析代码。
- **后端与 Agent 层**：FastAPI 结合 LangGraph 和 LangChain 构建 Agentic RAG 流程，既能满足异步请求，又易于扩展。通过 DAG 状态机可精准控制任务流转，确保教学任务的严谨性。
- **RAG 与知识库**：ChromaDB 适合作为轻量向量库，配合 PDF 文档切分和存储；进一步可引入 Neo4j 存储知识点和先决关系。
- **多模态生成**：Presenton 和 Mermaid.js 提供成熟的 PPT 和图表生成能力。Seedance 2.0 视频生成模块计算成本高，可作为演示选项。

- **开发效率**：iFlyCode IDE 插件能自动生成代码和测试样板，提高开发者效率。

四、系统总体架构

下面的文字架构图描述了系统的整体层次与通信关系：



各层说明：

1. **前端层**：采用 LobeChat 作为聊天界面。用户通过浏览器与系统交互，系统将 PPT 链接、思维导图等以卡片形式显示。前端通过 REST 或 WebSocket 调用后端接口，并支持语音输入/输出。
2. **后端 API 层**：使用 FastAPI 编写 REST 接口，处理登录注册、文件上传、会话管理和发起代理任务。后端负责将请求转发给 Agent 层，并将生成结果返回前端。
3. **Agent 编排层**：核心逻辑由 LangGraph 实现。根据用户请求，Tutor Agent 负责解析意图并拆解任务；Informer Agent 执行知识检索；Verifier Agent 验证答案；Practice Agent 调用资源生成工具；Insight Agent 分析用户画像。
4. **RAG 知识库层**：使用 ChromaDB 存储教材内容的嵌入向量；通过 LangChain 检索相似文本并返回结果。后续可引入 K12EduKG 及 Neo4j 构建知识图谱。
5. **多模态生成层**：集成 Presenton 生成 PPT、Mermaid.js 生成思维导图、Exam-Quiz-Test-PRO 生成测验题；可根据需要调用 Sparkgen 或 Seedance 生成数字人或实验视频。
6. **数据库层**：PostgreSQL/SQLite 用于存储用户、课程、生成记录、测试结果等关系型数据；Neo4j 用于存储学生画像与知识先决关系。Redis 用于缓存会话状态和异步任务。
7. **文件存储层**：使用 Minio 或本地文件系统保存上传的资料和生成的 PPT/Videos；文件路径存入数据库。
8. **用户画像/状态管理层**：主要存储学生画像、学习进度和测验历史，通过 Insight Agent 和数据库实现动态更新。
9. **权限与登录模块**：基于 FastAPI 实现 JWT 认证，区分学生、教师、管理员角色；LobeChat 也提供简单的访客访问权限控制。
10. **部署层**：通过 Docker Compose 启动各个服务，使用 Nginx 进行反向代理；配置 HTTPS；在云服务器上部署以确保外网访问。

五、模块拆解

每个模块包括目标、输入输出、所用开源项目、自定义代码等。开发难度按低/中/高划分，优先级按 1-3 表示优先顺序。

模块名称	模块目标	输入	输出	使用的开源项目	需要自己写的代码	关键接口/函数
用户登录与学生信息模块	实现注册、登录、角色鉴权，学生/教师信息管理	用户注册信息、登录凭证	JWT token、用户信息	FastAPI、PostgreSQL	编写接口、JWT 中间件、数据模型	<code>/api/auth/register</code> , <code>/api/auth/login</code>
学习资料上传模块	教师上传课程 PDF/Word，系统切分并存储	文件（二进制）	文件存储路径、文档切片	FastAPI、pdfminer、ChromaDB	文件解析、切片、嵌入存储逻辑	<code>/api/courses/upload</code> <code>ingest_document()</code>

模块名称	模块目标	输入	输出	使用的开源项目	需要自己写的代码	关键接口/函数
课程知识库构建模块	从上传资料中构建向量索引，生成初步知识图谱	文档切片、嵌入向量	向量索引、先决关系图（可选）	ChromaDB、LangChain、K12EduKG（可选）	调用嵌入 API、索引构建函数	<code>build_vector_store()</code> <code>extract_concepts()</code>
RAG 问答模块	对学生提问进行检索增强生成，返回带出处的答案	问题文本、会话上下文	答案文本、引用片段	LangGraph、LangChain、Spark LLM	编写 LangGraph 节点、Verifier 逻辑	<code>answer_question()</code> ， <code>verify_answer()</code>
学生画像模块	从对话和测验中提取学生特征，更新画像	对话历史、测验结果	六维画像 JSON	Neo4j、Spark LLM	Prompt 设计、抽取逻辑、图数据库更新	<code>update_profile()</code> ， <code>get_profile()</code>
Agent 调度模块	实现 Tutor/Informer/Verifier/Practice/Insight 智能体的编排	用户意图、文档、画像	任务结果	LangGraph	实现多节点 DAG 流程、状态传递	<code>run_workflow()</code> ， <code>router()</code>
个性化学习资源生成模块	根据知识点和画像生成讲义、思维导图、测验题等	知识点、学习画像	PPTX 文件、Mermaid 代码、题库 JSON	Presenton、Mermaid.js、Spark LLM、Exam-Quiz-Test-PRO	编写调用 Presenton API、Mermaid 代码 Prompt 和测验生成逻辑	<code>generate_ppt()</code> ， <code>generate_mindmap()</code> ， <code>generate_quiz()</code>
PPT/文档/思维导图生成模块	作为个性化生成的子模块，专注文件生成	Markdown 大纲、模板参数	PPTX / Mermaid 代码	Presenton、Mermaid.js	调用 REST API、拼接 Markdown	同上
测验题生成与批改模块	创建自适应测验、批改并记录错题	题库 JSON、学生答案	成绩、错题分析	Exam-Quiz-Test-PRO、Spark LLM	组卷算法、批改函数	<code>create_quiz()</code> ， <code>submit_quiz()</code>

模块名称	模块目标	输入	输出	使用的开源项目	需要自己写的代码	关键接口/函数
学习路径推荐模块	根据知识图谱和画像规划学习顺序	画像 JSON、知识图	推荐路径列表	A* 或 RL 算法	路径计算逻辑、推荐 API	<code>recommend_path()</code>
前端展示模块	构建聊天 UI、资源卡片展示	API 返回内容	富文本、图表、卡片	LobeChat	配置 Spark API、前端部署	LobeChat 配置脚本
后台管理模块	管理课程资料、学生画像、生成记录	管理员请求	仪表盘、统计数据	Ant Design, FastAPI	开发后台界面、API	<code>/api/admin/*</code>
部署与演示模块	打包所有服务，支持本地和云端部署	Docker Compose 文件	可启动的系统实例	Docker Compose, Nginx	写 compose 文件、环境配置	<code>docker-compose.yml</code> , <code>entrypoint.sh</code>

六、开源项目整合与胶水代码方案

下面针对核心开源项目说明其在系统中的角色、部署方式、通信和胶水代码设计。

1. LangGraph + Spark LLM（Agent 编排与推理）

- **角色与部署：**LangGraph 运行在后端服务中，用于定义多个 Agent 节点并形成有向无环的工作流。Spark LLM 提供推理能力，通过 HTTP API 调用并注入 API 密钥。在 Docker 容器内运行 Python 代码即可。
- **通信方式：**使用 LangGraph 库中的 `Graph` 对象注册多个节点：Tutor Agent、Informer Agent、Verifier Agent 等。每个节点都是一个异步函数，输入为状态对象，输出为更新后的状态。
- **胶水代码：**

```
# 定义状态结构
class AgentState(BaseModel):
    question: str
    history: list[str]
    retrieved_docs: list[str] = []
    answer: str | None = None
    profile: dict | None = None

# Tutor Agent: 解析意图, 调用Informer
async def tutor_agent(state: AgentState) -> AgentState:
    # 拆解问题, 可根据关键字决定是否查询知识库
    state.history.append(state.question)
    return state
```

```

# Informer Agent: 检索知识库
async def informer_agent(state: AgentState) -> AgentState:
    docs = vector_db.similarity_search(state.question, k=3)
    state.retrieved_docs = docs
    return state

# Verifier Agent: 验证与生成答案
async def verifier_agent(state: AgentState) -> AgentState:
    prompt = f"请根据以下文档回答问题并严格引用: {state.retrieved_docs}" # 简化示例
    response = spark_llm.generate(prompt)
    state.answer = response.text
    return state

# 构建图
g = Graph()
g.add_node("tutor", tutor_agent)
g.add_node("informer", informer_agent)
g.add_node("verifier", verifier_agent)
g.add_edge("tutor", "informer")
g.add_edge("informer", "verifier")
g.set_entry_node("tutor")
result_state = await g.run(AgentState(question="什么是梯度下降?", history=[]))

```

- **失败重试机制：**在 Verifier Agent 中如果大模型评分低于阈值则修改检索词并重新执行 Informer Agent。
- **数据存储与前端展示：**检索片段与答案将写入数据库 `chat_messages` 表；前端从后端接口获取响应，并通过 LobeChat 的卡片渲染方式展示答案和引用。

2. ChromaDB 与 LangChain (RAG 层)

- **角色与部署：**ChromaDB 用于存储教材切片的向量；部署在后端的独立进程或同一容器中。LangChain 负责加载文档、切分、嵌入和检索。
- **调用方式：**通过 Python SDK，如 `Chroma.from_documents()` 创建索引，并实现 `similarity_search()` 方法用于检索。
- **胶水代码职责：**编写文档加载函数，调用 Spark LLM 或第三方嵌入模型生成向量；将文档元数据（课程、页码）存入 `knowledge_chunks` 表；实现检索接口供 Informer Agent 调用。
- **输入输出格式：**输入为文本或用户问题；输出为与问题相似的文本片段列表。
- **失败重试机制：**若检索结果为空，可调整查询关键字或扩展检索范围。

3. Presenton / Mermaid.js / Exam Quiz Test PRO (多模态生成)

- **Presenton：**
- **部署：**Docker 容器运行，提供 REST API `/api/v1/ppt/presentation/generate`；将服务地址与 API 密钥配置在后端。
- **通信方式：**后端通过 HTTP POST 请求发送 Markdown 大纲、模板样式参数，返回 PPTX 和 PDF 文件链接。
- **胶水代码：**编写 `generate_ppt(markdown_outline: str) -> str` 函数，构建请求体并解析响应；保存生成的 PPTX 文件到 Minio，并记录到 `generated_resources` 表。
- **前端展示：**LobeChat 在聊天窗侧边卡片显示下载链接。
- **Mermaid.js：**

- **部署**：前端直接引入 Mermaid.js；无需后台部署。
- **调用方式**：在大模型的系统 Prompt 要求输出 Mermaid 代码块；后端将此代码作为一段文本发送给前端。
- **胶水代码**：在 Tutor/Practice Agent 响应中检测到包含 `mermaid` 标签的文本时，将其封装为一个 Artifact 类型的消息。LobeChat 自动渲染该代码为思维导图。
- **Exam Quiz Test PRO**：
 - **部署**：前端引用该 JS 库以渲染测验界面；题库 JSON 由 Practice Agent 生成。
 - **通信方式**：后端生成题库 JSON 对象，并通过 API 返回给前端；前端加载题库后自动渲染测验。
 - **胶水代码**：编写 `generate_quiz(chapter_id, difficulty)` 函数，在 Spark LLM 系统 Prompt 中要求输出题目和答案 JSON；保存测验数据到数据库。

4. LobeChat（前端层）

- **角色**：负责对话界面、文件上传表单、图片/视频卡片展示和语音功能。前端与后端通过 REST API 和 WebSocket 通信。
- **部署与配置**：使用官方 Docker 镜像，在 `docker-compose.yml` 中配置端口和访问密码。在“AI Providers”面板填入科大讯飞 APP ID 和密钥即完成模型接入。
- **胶水代码**：无需开发框架本身，但需编写配置文件及自定义内容卡片模板。前端可调用后端接口，如 `/api/agent/ask`、`/api/resource/download`。

5. FastAPI（后端 API）

- **角色**：提供 RESTful API，用于用户登录注册、资料上传、触发 Agent 流程、查询生成结果、管理后台。
- **部署**：在 Docker 镜像中运行 Uvicorn；使用 `--reload` 开发模式或 `--workers` 生产模式。
- **胶水代码主要职责**：
 - 定义 Pydantic 模型表示请求/响应数据结构；
 - 编写路由函数，调用相应的 Agent、数据库或文件存储服务；
 - 处理权限校验和异常捕获；
 - 在上传接口解析文件并调用文档切片函数；
 - 调用 Celery 任务完成耗时的 PPT/视频生成。

示例 FastAPI 路由框架：

```
@router.post("/courses/upload")
async def upload_course(file: UploadFile = File(...), current_user: User = Depends(get_current_user)):
    # 保存原始文件
    path = await save_file(file)
    # 解析并入库
    chunks = await ingest_document(path)
    return {"message": "uploaded", "chunks": len(chunks)}

@router.post("/agent/ask")
async def ask_question(request: AskRequest, current_user: User = Depends(get_current_user)):
    state = AgentState(question=request.question, history=request.history or [])
    result_state = await graph.run(state)
    return {"answer": result_state.answer, "refs": result_state.retrieved_docs}
```

```

@router.post("/resource/ppt")
async def create_ppt(req: PptRequest):
    task_id = celery_app.send_task("tasks.generate_ppt", args=[req.outline])
    return {"task_id": task_id}

@router.get("/resource/ppt/{task_id}")
async def get_ppt_result(task_id: str):
    result = AsyncResult(task_id)
    if result.ready():
        path = result.get()
        return FileResponse(path)
    return {"status": "processing"}

```

6. 数据存储和文件管理

- **PostgreSQL / SQLite**: 定义 ORM 模型（如 `User` , `Course` , `KnowledgeChunk` , `Quiz` , `GeneratedResource`）；通过 SQLAlchemy 与 FastAPI 集成；保证字段和索引合理。
- **Neo4j**: 用于保存学生画像及知识图谱；通过 Python `neo4j` 驱动执行 Cypher 查询与更新；胶水函数包括 `create_or_update_profile()` 和 `get_prerequisites(concept)`。
- **Minio / 本地文件系统**: 封装文件存储服务，实现 `save_file(file: UploadFile) -> str` 和 `get_file(path: str) -> StreamingResponse`。文件路径与元数据需要写入数据库。

7. 用户画像对生成的影响

在 Agent 流程中，Tutor Agent 会读取学生画像（从 `student_profiles` 表或 Neo4j 获取）。根据画像中的认知风格、知识掌握度等维度调整 Prompt，例如增加“使用思维导图解释”或“提供更多例题”；Practice Agent 生成资源时根据画像指定模板（简明/详细）或题目难度等级。这种动态调节能够真正实现个性化推荐。

七、核心业务数据流

下面描述核心流程。

流程一：教师上传课程资料 → 自动构建知识库

1. **用户操作**: 教师在前端选择课程并上传教材 PDF/Word。
2. **前端请求**: 调用 `/api/courses/upload` 上传文件。
3. **后端接口**: 保存文件至 Minio；调用文档解析函数，利用 PDFMiner 进行版面分析，拆分为段落；用 Spark LLM 或其他模型生成嵌入向量并存入 ChromaDB。
4. **Agent 动作**: 不涉及。
5. **数据库读写**: 向 `courses` , `course_files` 和 `knowledge_chunks` 表插入记录；向 ChromaDB 写入向量和元数据。
6. **文件读写**: 保存原始文件及分片内容。
7. **返回结果**: 返回上传成功信息和切分数量。
8. **异常处理**: 若文件格式不支持则返回错误码；若解析失败则记录日志。

流程二：学生提问 → RAG 检索 → Agent 组织答案 → 前端展示

1. **用户操作**: 学生在聊天窗口输入问题。

2. **前端请求**：调用 `/api/agent/ask`，传递问题文本和会话历史。
3. **后端接口**：FastAPI 接收请求并初始化 LangGraph 状态；调用 `graph.run(state)`。
4. **Agent 动作**：
 5. Tutor Agent 判断是否需要查询知识库；
 6. Informer Agent 调用 ChromaDB 检索出相关文档片段；
 7. Verifier Agent 将片段和问题提交给 Spark LLM 生成答案，并进行交叉验证；必要时回滚并重新检索。
 8. **数据库读写**：写入 `chat_messages`，记录问题、答案、引用段落；可更新学生画像。
 9. **文件读写**：无。
 10. **返回结果**：返回答复和引用；前端通过 LobeChat 将文本渲染在左侧，将思维导图或其他 Artifacts 渲染在右侧。
 11. **异常处理**：若知识库检索为空则提示“资料不足”；若大模型评分低则返回“需要更多信息”。

流程三：学生完成测验 → 系统分析错题 → 更新学生画像

1. **用户操作**：学生在前端测验页面作答；点击提交。
2. **前端请求**：调用 `/api/quiz/submit`，传递答案列表。
3. **后端接口**：批改答案，计算得分，并生成错题列表；将成绩存入 `quiz_attempts` 表。
4. **Agent 动作**：Insight Agent 分析错题对应知识点，更新学生画像的知识掌握度维度。
5. **数据库读写**：更新 `student_profiles`；可更新 Neo4j 关系；记录测验数据。
6. **文件读写**：无。
7. **返回结果**：返回分数、正确答案解析；前端显示结果并反馈学习建议。
8. **异常处理**：题目格式不合法或提交超时需提示。

流程四：学生请求个性化学习资源 → Agent 生成 PPT/思维导图/测验/路径

1. **用户操作**：学生点击“生成讲义”或“生成思维导图”。
2. **前端请求**：调用 `/api/resource/generate`，带上知识点、资源类型。
3. **后端接口**：FastAPI 向 Practice Agent 发送请求；如果为 PPT 则通过 Celery 异步任务调用 Presenton；如果为思维导图则在 Prompt 中要求输出 Mermaid 代码；测验生成类似。
4. **Agent 动作**：Practice Agent 协调调用 Presenton、Mermaid.js、Exam Quiz Test PRO 等工具；需要时向 Spark LLM 请求标题、脚本等文本。
5. **数据库读写**：存储生成记录到 `generated_resources`；将测验题库写入 `quizzes`。
6. **文件读写**：保存生成的 PPTX 和其他文件至 Minio；返回文件 URL。
7. **返回结果**：返回资源链接或嵌入代码；前端渲染卡片或提供下载按钮。
8. **异常处理**：若资源生成失败则返回错误信息；可重试或降级为文本说明。

流程五：管理员查看课程、学生画像、生成记录和反馈数据

1. **用户操作**：管理员登录后台页面，查看课程列表、学生画像统计、生成资源列表。
2. **前端请求**：调用 `/api/admin/*` 系列接口：课程数据、用户数据、日志查询等。
3. **后端接口**：查询数据库并返回汇总数据；可以使用 SQL 聚合或 Neo4j 查询。
4. **Agent 动作**：无。
5. **数据库读写**：读取 `courses`，`student_profiles`，`generated_resources`，`system_logs`；不修改数据。
6. **文件读写**：如需下载 PPT 或视频，通过文件存储获取路径并提供下载。
7. **返回结果**：返回数据列表和统计结果；前端以表格或图表展示。

8. 异常处理：无。

八、数据库设计

为了满足功能需求，建议使用关系型数据库（如 PostgreSQL）与图数据库（Neo4j）结合。下表描述了主要表结构。

表名	字段	类型	作用	必填	关联关系
users	id (PK)	UUID	用户标识	是	-
	username	varchar	登录名	是	-
	password_hash	varchar	密码哈希	是	-
	role	enum('student','teacher','admin')	角色	是	-
	created_at	timestamp	创建时间	是	-

| **courses** | id (PK) | UUID | 课程标识 | 是 | - | | name | varchar | 课程名称 | 是 | - | | teacher_id | UUID (FK) | 关联教师 | 是 | users.id | | description | text | 课程简介 | 否 | - | | created_at | timestamp | 创建时间 | 是 | - |

| **course_files** | id (PK) | UUID | 文件标识 | 是 | - | | course_id | UUID (FK) | 对应课程 | 是 | courses.id | | file_name | varchar | 文件名 | 是 | - | | file_path | varchar | 存储路径 | 是 | - | | uploaded_at | timestamp | 上传时间 | 是 | - |

| **knowledge_chunks** | id (PK) | UUID | 文档切片 | 是 | - | | course_id | UUID (FK) | 关联课程 | 是 | courses.id | | content | text | 文本片段 | 是 | - | | vector_id | varchar | 在 ChromaDB 中的键 | 是 | - | | page_number | int | 原始页码 | 否 | - |

| **student_profiles** | id (PK) | UUID | 画像记录 | 是 | - | | user_id | UUID (FK) | 学生 | 是 | users.id | | knowledge_state | jsonb | 知识掌握度 | 是 | - | | cognitive_style | varchar | 认知风格 | 是 | - | | pace | varchar | 学习节奏 | 是 | - | | path_preference | varchar | 路径偏好 | 是 | - | | meta_learning | varchar | 元学习能力 | 是 | - | | emotion | varchar | 情感/性格 | 是 | - | | updated_at | timestamp | 更新时间 | 是 | - |

| **chat_sessions** | id (PK) | UUID | 会话标识 | 是 | - | | user_id | UUID (FK) | 学生 | 是 | users.id | | created_at | timestamp | 创建时间 | 是 | - |

| **chat_messages** | id (PK) | UUID | 消息标识 | 是 | - | | session_id | UUID (FK) | 会话 | 是 | chat_sessions.id | | sender | enum('user','agent') | 发送方 | 是 | - | | content | text | 消息内容 | 是 | - | | refs | jsonb | 引用片段 | 否 | - | | created_at | timestamp | 时间 | 是 | - |

| **generated_resources** | id (PK) | UUID | 生成记录 | 是 | - | | user_id | UUID (FK) | 生成者 | 是 | users.id | | type | enum('ppt','mindmap','quiz','video','doc') | 资源类型 | 是 | - | | resource_path | varchar | 文件或代码路径 | 是 | - | | metadata | jsonb | 生成参数 | 否 | - | | created_at | timestamp | 时间 | 是 | - |

| **quizzes** | id (PK) | UUID | 题库标识 | 是 | - | | course_id | UUID (FK) | 关联课程 | 是 | courses.id | | json_content | jsonb | 题目和答案 | 是 | - | | created_at | timestamp | 时间 | 是 | - |

| **quiz_attempts** | id (PK) | UUID | 测验结果 | 是 | - | | quiz_id | UUID (FK) | 关联题库 | 是 | quizzes.id | | |
user_id | UUID (FK) | 学生 | 是 | users.id | | | score | float | 得分 | 是 | - | | answers | jsonb | 学生答案 | 是 | - | |
created_at | timestamp | 时间 | 是 | - |

| **agent_tasks** | id (PK) | UUID | Agent 流程记录 | 是 | - | | user_id | UUID (FK) | 关联用户 | 是 | users.id | | |
task_type | varchar | 任务类型 | 是 | - | | status | enum('pending','running','success','failed') | 状态 | 是 | - | |
result | jsonb | 返回内容 | 否 | - | | created_at | timestamp | 时间 | 是 | - |

| **system_logs** | id (PK) | UUID | 日志标识 | 是 | - | | level | varchar | 日志级别 | 是 | - | | message | text | 日志
信息 | 是 | - | | timestamp | timestamp | 时间 | 是 | - |

数据结构说明： `student_profiles` 表可扩展字段以支持更多维度。由于 Neo4j 擅长存储图关系，可通过 `student_id` 与知识概念之间建立关系，用于学习路径算法。

九、项目目录结构

推荐的目录结构示例如下：

```
project-root/
├── frontend/                # LobeChat 配置文件及自定义组件
├── backend/
│   ├── app/
│   │   ├── api/            # FastAPI 路由定义
│   │   ├── agents/         # LangGraph 节点、Agent 定义
│   │   ├── services/       # 服务层，如文件存储、向量检索、资源生成
│   │   ├── models/         # Pydantic/ORM 模型
│   │   ├── schemas/        # 数据请求/响应模型
│   │   ├── tasks/          # Celery 任务定义
│   │   ├── core/           # 配置、初始化数据库、日志
│   │   ├── utils/          # 工具函数（分词、嵌入等）
│   │   └── main.py         # 应用入口，创建 FastAPI 实例
│   └── Dockerfile          # 后端镜像构建文件
├── data/
│   ├── raw/                # 上传的原始文件
│   └── processed/          # 分片后的文本
├── scripts/                # 脚本，如初始化数据库、填充数据
├── docker-compose.yml      # 定义多个服务
├── docs/                   # 项目文档和流程图
└── README.md
```

这样的分层有利于团队协作：`api` 负责接口；`agents` 集中管理智能体逻辑；`services` 处理业务；`tasks` 处理异步任务；`models` 定义 ORM；`utils` 封装公共函数；`frontend` 放置 LobeChat 配置。

十、API 接口设计

以下给出核心 API 设计，包括 URL、方法、参数、返回等。

接口名称	URL	方法	请求参数	返回参数	调用服务	错误码	示例 JSON
用户注册	/api/auth/register	POST	{username, password, role}	{id, username}	用户服务	400 参数错误; 409 用户存在	{ "id": "uuid", "username": "alice"} }
用户登录	/api/auth/login	POST	{username, password}	{token, role}	用户服务	401 未授权	{ "token": "jwt", "role": "student"} }
上传课程资料	/api/courses/upload	POST (multipart)	file: PDF/Doc	{message, chunks}	文件上传服务	400 文件类型错误	{ "message": "uploaded", "chunks": 120} }
创建知识库	/api/courses/{course_id}/build	POST	{course_id}	{status}	文档切分 + 向量化	404 课程不存在	{ "status": "success"} }
学生提问	/api/agent/ask	POST	{question, history[]}	{answer, refs[]}	Agent 服务	500 内部错误	{ "answer": "...", "refs": ["doc1#p3", "doc2#p5"]} }
生成学习资源	/api/resource/generate	POST	{type, outline, options}	{task_id}	Practice Agent + Celery	400 参数错误	{ "task_id": "abc123"} }
获取生成结果	/api/resource/{task_id}	GET	-	{status, url/code}	Celery + 文件服务	404 任务不存在	{ "status": "finished", "url": "/files/xyz.pptx"} }
创建测验	/api/quiz/create	POST	{course_id, chapter_id}	{quiz_id}	Practice Agent	404 课程不存在	{ "quiz_id": "uuid"} }

接口名称	URL	方法	请求参数	返回参数	调用服务	错误码	示例 JSON
提交测验	/api/quiz/submit	POST	{quiz_id, answers[]}	{score, mistakes}	Quiz 服务	400 参数错误	{"score": 80, "mistakes": [1,4]}
获取学生画像	/api/profile/{user_id}	GET	-	{profile}	Insight Agent	404 用户不存在	{"knowledge_state": {...}, "cognitive_style": "visual"}
获取学习路径	/api/path/recommend	POST	{user_id, course_id}	{path[]}	路径推荐服务	400 参数错误	{"path": ["Chapter 1", "Chapter 3"]}
管理后台接口	/api/admin/overview	GET	-	各类统计	管理服务	403 权限不足	{"courses": 3, "students": 100}

十一、4 周开发路线

项目开发分为三版本：最小 Demo、比赛可展示版本和增强版本。表格以 4 周为周期分解任务。

周次	本周目标	每个人任务（3 人团队示例）	必须完成功能	可选功能	验收标准	风险与解决
第 1 周	技术验证、环境搭建	前端负责人：部署 LobeChat，配置 Spark API，完成登录界面； 后端负责人：搭建 FastAPI 项目框架、用户模型、JWT 认证；Agent 负责人：学习 LangGraph、实现简单的 Tutor+Informer+Verifier 流程	基础用户注册登录、文件上传接口、基本问答流程	安装 Celery；配置 ChromaDB；初步文档切分	能成功登录并通过一个简单问题得到答案，回答引用文档	熟悉技术栈时间不足，可借助 iFlyCode 快速生成样板代码； Spark API 配置出错可查官方文档

周次	本周目标	每个人任务（3人团队示例）	必须完成功能	可选功能	验收标准	风险与解决
第2周	打通核心链路	前端：接入学生提问、展示答案和引用；后端：完成文档解析与向量索引构建、RAG 检索；Agent 负责人：完善 LangGraph 流程，加入失败重试机制	上传教材并构建知识库；问答接口返回引用；保存对话历史	初步学生画像提取（采用简单统计）	学生可上传课程材料并向系统提问，系统能够给出引用明确的答案	文档解析可失败；可适当减少切分粒度；若向量匹配效果差，采用更大 k 或使用搜索词改写
第3周	多模态生成与学生画像	前端：集成 Mermaid 渲染和测验页面，展示生成的 PPT 下载链接；后端：实现 Presenton 调用和测验生成，完成学生画像更新；Agent 负责人：新增 Practice Agent 和 Insight Agent	PPT / 思维导图 / 测验生成；更新学生画像	视频生成（调用 Sparkgen 或 Seedance）	成功生成 PPT 和思维导图并在 LobeChat 中展示；学生做完测验后画像发生更新	Presenton 调用失败可重试；暂时用固定模板填充 PPT；图数据库连接缓慢可暂用 JSON 存储画像
第4周	部署、优化与答辩准备	前端：完善后台管理页面、统计图；后端：完成学习路径推荐、日志与监控；Agent 负责人：调优 Prompt、完善验证逻辑；团队：撰写文档和演示脚本	学习路径推荐；管理后台；系统部署脚本	情感计算和语音交互	系统可在云服务器稳定运行；管理员可查看课程和生成记录；答辩时可展示完整链路	服务器性能不足可采用小模型或缩小数据规模；知识图谱算法可用简单的先决关系图代替强化学习

十二、团队分工

针对三人或四人团队给出任务分配。

三人团队

- 前端与产品展示负责人**
- 负责模块：**LobeChat 部署与配置、UI 自定义、前端上传与展示页面、后台管理界面。
- 每周任务：**第 1 周部署 LobeChat 和登录界面；第 2 周对接问答和知识库上传功能；第 3 周集成 PPT/思维导图/测验显示；第 4 周完善后台管理、做答辩 PPT 和演示脚本。
- 学习技术：**React/Next.js、LobeChat 配置、Mermaid 渲染、Docker 部署。
- 最终交付物：**前端源码、定制主题、演示视频界面。
- 接口边界：**与后端工程师约定 REST API；不直接访问数据库。
- 风险点：**前端样式问题可依赖 LobeChat 默认样式；如需修改深层逻辑，应谨慎阅读源代码。

8. 后端与数据库负责人

9. **负责模块**：FastAPI 项目、用户认证、文件上传、数据库模型、知识库构建、资源生成任务、管理后台 API。

10. **每周任务**：第 1 周实现用户注册登录、数据库连接；第 2 周完成文件解析、嵌入存储；第 3 周实现生成资源接口和测验模块；第 4 周开发路径推荐算法、日志和监控。

11. **学习技术**：FastAPI、SQLAlchemy、ChromaDB、Celery、Minio、Docker Compose。

12. **最终交付物**：后端源码、数据库迁移脚本、API 文档、docker-compose 文件。

13. **接口边界**：与 Agent 负责人传递请求数据和返回结果；与前端只通过 API 交互。

14. **风险点**：大模型 API 可能不稳定，要设置重试；文件解析耗时长，应优化批处理。

15. Agent/RAG/多模态负责人

16. **负责模块**：LangGraph 智能体设计、Spark LLM 接入、知识检索、Prompt 工程、Presenton 调用、图数据库设计与分析。

17. **每周任务**：第 1 周熟悉 LangGraph 并实现初版 Tutor+Informer+Verifier；第 2 周调优检索逻辑和失败重试；第 3 周新增 Practice/Insight Agent 和资源生成；第 4 周完善知识图谱、学生画像更新与路径推荐。

18. **学习技术**：LangGraph、LangChain、Spark API、Presenton API、Mermaid 语法、Neo4j。

19. **最终交付物**：Agent 代码、Prompt 模板、运行示例、改进策略文档。

20. **接口边界**：向后端暴露函数 `ask_question`，`generate_resource`；不直接处理 Web 请求或数据库事务。

21. **风险点**：Prompt 效果不稳定需迭代；图数据库学习曲线高可先采用 JSON 存储。

四人团队

在三人团队基础上新增：

1. 多模态生成/部署/答辩材料负责人

2. **负责模块**：Presenton 和视频生成服务部署、多媒体资源测试、Docker Compose 配置、系统整体部署、答辩材料准备（PPT、视频录制）。

3. **每周任务**：第 2 周部署 Presenton；第 3 周测试 Seedance 或 Sparkgen；第 4 周完善部署脚本、准备演示视频和答辩 PPT。

4. **学习技术**：Docker、Presenton API、Seedance、视频剪辑；熟悉云服务器部署。

5. **接口边界**：提供生成资源的 API 给后端使用；负责部署稳定性和演示呈现。

6. **风险点**：视频生成成本高可降级；服务器资源不足时可选择 PPT 演示替代。

十三、部署方案

1. 本地 Docker Compose 部署

编写 `docker-compose.yml` 以启动以下服务：

- **backend**: 基于 Python3.10 的镜像，运行 FastAPI 应用；安装 requirements；挂载 `data/` 与 `logs/`；暴露端口 8000。
- **frontend**: 使用官方 `lobehub/lobe-chat` 镜像；设置环境变量 `ACCESS_CODE` 用于管理后台密码；暴露端口 3210。
- **chroma**: 运行官方 ChromaDB 镜像；持久化数据卷。
- **postgres**: 运行 PostgreSQL；设置用户、密码、数据库名称；挂载数据卷。
- **redis**: 用于 Celery 任务队列。
- **celery_worker**: 使用 backend 镜像作为基础，启动 Celery worker 进程。
- **minio**: 对象存储服务；配置访问密钥；设置端口 9000。
- **neo4j** (可选)：图数据库；配置内存；暴露端口 7687。
- **nginx** (可选)：作为反向代理；实现 HTTPS 和路径分发，例如 `/api` 转向 backend，其他访问转向 frontend。

Docker Compose 脚本中定义服务间依赖，使用 `.env` 文件管理环境变量（Spark APP ID 等）。

2. 云服务器部署

在云服务器（如 阿里云、腾讯云）配置安全组开放端口 80/443/3210/8000；上传 `docker-compose.yml` 和 `.env`；运行 `docker compose up -d`。建议使用 Nginx 配置 HTTPS 证书。对于科大讯飞 API 凭证，使用环境变量注入，避免泄露。

3. 稳定性保障

- 使用 `restart: always` 保证容器异常退出自动重启；
- 对长时间任务采用 Celery 后台队列；
- 为 Spark API 调用设置超时和重试次数；
- 监控容器资源占用，必要时调整服务实例。

十四、风险与降级方案

风险	影响	预防方案	降级方案	最低可接受效果
开源项目部署困难	无法快速上线	提前在本地验证部署脚本；选用文档完善的项目；	如 Presenton 部署失败，可改用 pptxgenjs 生成简易 PPT；如 ChromaDB 配置困难，可改用 SQLite 存嵌入	至少能生成 Markdown 格式讲义和思维导图
大模型 API 不稳定或额度耗尽	无法获取答案或资源	配置合理重试与回退机制；监控调用量；	当 Spark LLM 不可用时，可切换到开放源 code 模型（如 ChatGLM）以应急；限制每天使用次数	系统至少提供基于知识库的搜索结果摘要

风险	影响	预防方案	降级方案	最低可接受效果
RAG 效果不好	回答不准确、幻觉	使用 Agentic RAG 多步检索与验证机制；调整嵌入模型和分片大小；	如果检索仍不准确，可直接返回“资料不足，请查看课本第 X 页”并附上原文；利用提示模板限制模型自由发挥	用户能看到原始文档片段并自行阅读
多模态生成速度慢	长等待影响体验	使用 Celery/Redis 异步队列；在前端提供生成进度条或提示	对于视频生成，提供 PPT + 静态图替代；初赛阶段可不生成视频	能生成 PPT 或思维导图即可
视频生成成本高	GPU 需求超预算	不在初赛阶段使用 Seedance；使用 Sparkgen 生成简短数字人视频；	取消视频功能，改为生成讲义和思维导图	系统提供文本和图表即可
前端展示不完整	影响用户体验	使用成熟的 LobeChat 减少自定义 UI；提前规划卡片样式	如卡片功能无法实现，可用简单链接或文本替代；	能展示回答和引用即可
数据库设计复杂	维护困难	采用渐进式 schema；初期先用简单表；	如果时间紧，可以不使用 Neo4j，直接在 PostgreSQL 中用 JSON 存储画像；	能保存用户、课程、聊天和生成记录即可
团队时间不够	功能缺失	按优先级开发；MVP 阶段只实现问答、资料上传和基本生成功能；	舍弃高难度功能，如强化学习路径推荐、视频生成等；	系统至少能上传课程、回答学生问题并生成 PPT/思维导图

十五、最终交付清单

- 源代码：**包含前端 LobeChat 配置、后端 FastAPI 服务、LangGraph 智能体逻辑、Celery 任务、Docker 配置。
- 部署文档：**详细说明安装依赖、配置环境变量、运行 Docker Compose、开放端口、Nginx 配置等。
- 演示账号：**学生账号与教师账号各一个，用于评审登录测试；提供管理员账号访问后台。
- 测试数据：**示例课程（例如《人工智能导论》）的 PDF/Word 文件以及生成的分片数据；示例用户画像。
- 课程知识库样例：**包含切分的文档片段及其向量索引；配套说明如何构建索引。
- PPT 生成样例：**根据课程大纲生成的 PPT 文件，展示 Presenton 效果。
- 思维导图样例：**由 Spark LLM 输出的 Mermaid 代码片段及其渲染效果。
- 测验样例：**生成的题库 JSON 和测验界面截图。
- 学生画像样例：**包含六维指标的 JSON 文件，展示如何从对话/测验更新画像。
- 系统架构图：**文字架构图或简单示意图，展示各模块和数据流。
- 答辩 PPT：**介绍项目目标、技术路线、实现过程、演示效果、创新点等。
- 演示视频：**约 5-7 分钟，完整展示系统主要功能、上传资料、问答、生成资源及后台管理；可由 Seedance 或 Sparkgen 生成简短开场视频加强效果。
- 项目说明文档：**描述需求分析、功能设计、技术选型、使用方法及注意事项。
- 技术创新点说明：**强调 Agentic RAG、防幻觉机制、多模态生成、个人画像与路径规划等创新点。
- 开源项目引用说明：**列出所用开源项目及其许可证，符合论文和比赛要求。

十六、最推荐的实际落地路线

为了让大学生团队在有限时间内做出可演示的成品，建议采用如下步骤：

1. **确定核心功能与范围**：聚焦于对话式问答、课程资料上传、知识库检索、思维导图生成。暂时放弃高成本的多模态视频和复杂图谱构建。
2. **快速部署前端**：使用官方 LobeChat 镜像在本地或服务器拉起前端，配置科大讯飞 API 密钥，即可获得成熟聊天 UI。此步骤无需写代码，节约大量时间。
3. **搭建 FastAPI 后端**：创建基本项目结构，完成用户认证和文件上传接口；通过 pdfminer 或 PyPDF 实现 PDF 切分；使用 Spark LLM 嵌入或外部嵌入模型生成向量，存入 ChromaDB。实现一个简单的 `ask_question` API，通过 LangChain 直接检索并调用模型生成回答。
4. **引入 LangGraph 实现 Agentic RAG**：在简易问答基础上，引入 LangGraph 将步骤拆解为 Tutor、Informer 和 Verifier；重点实现失败重试逻辑，保证回答来源可信。
5. **集成 Mermaid.js 生成思维导图**：调整 Prompt 要求模型输出 Mermaid 代码；前端自动渲染成图表。此功能易于实现且视觉效果好，是演示亮点。
6. **添加 PPT 生成**：配置 Presenton Docker 服务，编写简单的文本大纲生成函数；通过异步任务生成 PPT，并将链接在前端以卡片形式展示。若 Presenton 部署失败，可使用 pptxgenjs 生成简单 PPT。
7. **完善学生画像与测验**：在基本功能稳定后，引入测验生成器生成题库；记录学生答题情况并更新画像。可通过简单的统计模型代替复杂的图数据库。
8. **部署与演示**：使用 Docker Compose 打包服务；在云服务器部署并调试端口；准备演示视频和答辩 PPT，强调 Agentic RAG、思维导图和 PPT 生成的效果和创新性。

沿着以上路线，即使资源有限、时间紧张，也能构建出一个可用的系统原型，并在比赛中展示完整链路。进一步的功能可在正式版本中逐渐完善，如学习路径推荐、情感计算和高保真视频生成等。